CORE ARCHITECTURE THEORIES

5 Cara Berpikir dalam *if* Python

Bukan sekedar hafal syntax — ini framework berpikir untuk merancang percabangan yang solid, nggak bocor, dan bisa di-scale.

Russian Dolls	Nested Hierarchy
High-Voltage Filter	Elif Priority
Circuit Breaker	Boolean AND/OR
Flagging	Boolean Toggle
The Safety Net	Final Else

TEORI 1 · NESTED HIERARCHY

// The Gatekeeping Pattern

1

🪆 Russian *Dolls*

Lo nggak bakal ngecek detail kecil kalau kondisi besarnya belum terpenuhi.

🎯 **Karakteristik:** Vertikal & Dependen. Setiap if di dalam adalah "anak" dari if di luarnya. Kalau parent False, anak nggak pernah dicek.

listrik.py

```
# 3 lapisan — seperti boneka Rusia
pusat = True
gardu = True
token = 75

if pusat: # Gate 1 (terluar)
```

⏰ **KAPAN PAKAI**

Saat ada variabel yang jadi **Syarat Mutlak** bagi variabel lain. Contoh: sistem harus aktif dulu sebelum ngecek suhu, token, atau status lainnya.

⚠️ **Jangan terlalu dalam.** Lebih dari 3 level nested biasanya tanda kode butuh di-refactor. Indentasi makin masuk = makin susah dibaca.

TEORI 2 · ELIF PRIORITY CHAIN

TEORI 2 · ELIF PRIORITY CHAIN

// The Priority Pattern

⚡ High-Voltage *Filter*

Python itu malas — dia berhenti di detik pertama dia nemu jawaban True.

🎯 **Karakteristik:** Horizontal & Eksklusif. Hanya *satu* blok yang jalan. Urutan kondisi menentukan segalanya.

suhu_mac.py

```
suhu = 55

# Paling ekstrim SELALU di atas ↓
if suhu >= 60:
    print("🔥 Danger Zone!")
elif suhu >= 50:
    print("⚠️ Menuju HeatWave")
```

⏰ **KAPAN PAKAI**

Saat mengecek **satu variabel dengan banyak rentang nilai**. Kode di atas = cukup *suhu = 55*, langsung ke "Menuju HeatWave" tanpa cek sisanya.

🔑 **Aturan emas:** Nilai terbesar / kondisi paling kritis selalu di `if` pertama. Urutan salah → logic bocor.

TEORI 3 · BOOLEAN AND / OR

// The Combination Pattern

🔌 Circuit *Breaker*

3

Gabungkan banyak syarat dalam satu baris. Flat, ringkas, powerful.

🔗

AND

Semua saklar harus ON. Satu False = seluruhnya False.

⚡

OR

Satu saklar cukup. Satu True = seluruhnya True.

pln_token.py

```
sistem_pln = True
token_pln = 50

# AND: semua harus benar
if token_pln >= 100 and sistem_pln:
    print("🛢️ Penuh, Aman!")
elif token_pln >= 50 and sistem_pln:
    print("⚠️ Segera Isi Ulang!")
```

⏰ **KAPAN PAKAI**

Saat ada **beberapa variabel berbeda** yang harus dicek bersamaan tanpa hierarki. Lebih ringkas dari Nested If untuk kondisi flat.

TEORI 4 · BOOLEAN TOGGLE

// The Status Accumulator

🚦 Flagging

Pisahkan proses "pengecekan" dari "eksekusi". Status disimpan dulu di variabel bendera.

4

🏗️ **Cara kerja:** Set variabel Boolean sebagai "bendera" status. Cek di bagian atas, jalankan logika di bagian bawah. Kode jadi terstruktur dan mudah di-debug.

hujan_flag.py

```
hujan = True
punya_payung = True

# Fase 1: SET FLAG
bisa_jalan = False # bendera awal

if hujan and punya_payung:
   bisa_jalan = True
elif not hujan:
   bisa_jalan = True
```

⏰ **KAPAN PAKAI**

Saat logika pengecekan **kompleks tapi eksekusinya simpel**. Cocok untuk sistem dengan banyak kondisi masuk yang berujung ke satu keputusan akhir.

5

TEORI 5 · FINAL ELSE · ERROR HANDLING

// The Edge Case Guard

🛡️ The

Safety Net

else bukan cuma pajangan. Dalam sistem yang solid, ini adalah error handling pertama.

💡 **Definisi Edge Case:** Input tak terduga yang nggak masuk ke kondisi manapun. Data kotor dari sensor, user yang ngetik sembarangan, atau status yang nggak valid.

status_guard.py

```
status = "UNKNOWN" # data kotor!

if status == "ON":
    print("⚡ Sistem Aktif")
elif status == "OFF":
```

⏰ **KAPAN PAKAI**

Selalu. Setiap sistem yang menerima input dari luar (user, sensor, API) wajib punya Safety Net. Tanpa `else`, data buruk = program diam tanpa keterangan.

📌 **Pro tip:** Isi else dengan pesan error yang informatif — bukan cuma `"error"`. Sebutkan nilai yang bermasalah agar mudah di-debug.

Pilih teori yang *tepat*,

bukan yang termudah.

Framework ini bukan hafalan — ini cara lo membaca situasi sebelum nulis satu baris kode.

1 🪆 **Russian Dolls**
Ada syarat mutlak? Nested.

2 ⚡ **High-Voltage Filter**
Rentang nilai? Elif, ekstrim dulu.

3 🔌 **Circuit Breaker**
Multi-variabel flat? AND / OR.

4 🚦 **Flagging**
Pisahkan cek & eksekusi.

5 🛡️ **The Safety Net**
else selalu ada, isi dengan error msg.

📋 **SOP SEBELUM NULIS IF**

1 **Hierarki?** Nested Dolls
2 **Prioritas?** Elif Chain
3 **Master Switch?** If terluar
4 **Safety Net?** Else wajib